doesn’t eliminate the need for human feedback and evaluation, it produces much more complete results overall. And, while the iteration loop is time consuming, it’s time you could be drinking a coffee or working on other tasks.

AutoGen Studio is a tool developed by the AutoGen team that provides a helpful introduction to conversable agents. In the next exercise, we’ll install Studio and run some experiments to see how well the platform performs. These tools are still in a rapid development cycle, so if you encounter any problems, consult the documentation on the AutoGen GitHub repository.

4.1.1 Installing and using AutoGen Studio

Open the `chapter_04` folder in Visual Studio Code (VS Code), create a local Python virtual environment, and install the `requirements.txt` file. If you need assistance with this, consult appendix B to install all of this chapter’s exercise requirements.

Open a terminal in VS Code (Ctrl-`, Cmd-`) pointing to your virtual environment, and run AutoGen Studio using the command shown in listing 4.1. You’ll first need to define an environment variable for your OpenAI key. Because ports 8080 and 8081 are popular, and if you have other services running, change the port to 8082 or something you choose.

Listing 4.1 Launching AutoGen Studio

```
# set environment variable on Bash (Git Bash)
export OPENAI_API_KEY=”<your API key>”          #1

# sent environment variable with PowerShell
$env:VAR_NAME =”<your API key>"                #1

autogenstudio ui --port 8081     #2
```

#1 Use the appropriate command for your terminal type.
#2 Change the port if you expect or experience a conflict on your machine.

Navigate your browser to the AutoGen Studio interface shown in figure 4.3 (as of this writing). While there may be differences, one thing is for sure: the primary interface will still be chat. Enter a complex task that requires